United States Patent	12418398
Kind Code	B2
Date of Patent	September 16, 2025
Inventor(s)	Koskas; Michel et al.

Homomorphic encryption method and associated devices and system

Abstract

An homomorphic encryption technique enables one to carry on operations on encrypted messages without decrypting them, these encrypted messages being associated to unencrypted messages that belong to a discrete set consisting of p.sub.i distinct elements, with p.sub.i being an odd number, the elements of the discrete set being such that twice the difference between any two of these elements is not an integer number, said discrete set being for instance the discrete torus T.sub.pi={−(p.sub.i−1)/2, −(p.sub.i−1)/2+1, . . . , (p.sub.i−1)/2−1, (p.sub.i−1)/2}/p.sub.i. In particular, a specific bootstrapping procedure form such encrypted messages is disclosed. The disclosed technology concerns also an homomorphic encryption technique for carrying on operations on encrypted messages associated to unencrypted messages that belong Z.sub.p, p being equal to the product of r integer numbers p.sub.i, i=1 . . . r that are pairwise coprime. The disclosed technology concerns also associated electronic devices and systems implementing such techniques.

Inventors: **Koskas; Michel (Paris, FR), Chartier; Philippe (Rennes, FR), Lemou; Mohammed (Acigné, FR), Mehats; Florian (Rennes, FR)**

Applicant: **RAVEL TECHNOLOGIES** (Paris, FR)

Assignee: **RAVEL TECHNOLOGIES (Paris, FR)**

Family ID: **1000008822293**

Appl. No.: **18/268103**

Filed (or PCT Filed): **December 18, 2020**

PCT No.: **PCT/IB2020/001147**

PCT Pub. No.: **WO2022/129979**

PCT Pub. Date: June 23, 2022

Prior Publication Data

Document Identifier	Publication Date
US 20240064000 A1	Feb. 22, 2024

Publication Classification

Int. Cl.: **H04L9/00** (20220101)

U.S. Cl.:

CPC **H04L9/008** (20130101);

Field of Classification Search

CPC: H04L (9/008)

References Cited

U.S. PATENT DOCUMENTS

Patent No.	Issued Date	Patentee Name	U.S. Cl.	CPC
2023/0188318	12/2022	Paillier	380/28	H04L 9/008

OTHER PUBLICATIONS

TFHE: Fast Fully Homomorphic Encryption Over the Torus by Gama-Gama, published 2019. (Year: 2019). cited by examiner

International Search Report as issued in International Patent Application No. PCT/IB2020/001147, dated Sep. 13, 2021. cited by applicant

Chillotti, I., et al., "TFHE: Fast Fully Homomorphic Encryption over the Torus*," International Association for Cryptologic Research, vol. 20180510:205538, May 2018, XP061025748, Retrieved from the Internet: URL:http://eprint.iacr.org/2018/421.pdf [retrieved on May 8, 2018], pp. 1-59. cited by applicant

Okada, H., et al., "Integer-Wise Functional Bootstrapping on TFHE: Applications in Secure Integer Arithmetics," Advances in Intelligent Data

Analysis XIX; [Lecture Notes in Computer Science; Lect.Notes Computer], vol. 7, No. 499304, Nov. 2020, XP047572334, pp. 107-125. cited by applicant
Chillotti, I., et al., “Programmable Bootstrapping Enables Efficient Homomorphic Inference of Deep Neural Networks,” Oct. 2020, XP055837258, Retrieved from the Internet: URL:https://whitepaper.zama.ai/whitepaper.pdf> [retrieved on Sep. 2, 2021], 14 pages. cited by applicant
Halevi, S., et al., “An Improved RNS Variant of the BFV Homomorphic Encryption Scheme,” Advances in Databases and Information Systems; [Lecture Notes in Computer Science; Lect.Notes Computer], Feb. 2019, XP047502870, pp. 83-105. cited by applicant

Primary Examiner: Tran; Vu V

Attorney, Agent or Firm: Pillsbury Winthrop Shaw Pittman LLP

Background/Summary

CROSS-REFERENCE TO RELATED APPLICATIONS

(1) This application is the U.S. National Stage of PCT/IB2020/001147, filed Dec. 18, 2020. The content of this application is incorporated herein by reference in its entirety.

TECHNICAL FIELD

(2) The disclosed technology concerns methods, devices and systems for homomorphic encryption.

BACKGROUND

(3) Homomorphic encryption, which allows one to perform calculations or data processing on encrypted data, without decrypting it first, has attracted a lot of attention recently.

(4) Indeed, digital data processing of personal data has become ubiquitous in our everyday lives. Confidentiality and privacy protection of such data has thus become critical, as such personal data tends to circulate more and more in our digital environment.

(5) In this context, homomorphic encryption is a very promising solution, as it enables to process data while preserving very securely data anonymization and privacy at the same time, as the data is not decrypted during its processing.

(6) Homomorphic encryption is usually based on the “learning with error” encryption scheme, in which the encrypted message c=(a,b) is derived from the unencrypted message m according to the following formula: b=m+e+a.Math.s, where: s is a secret key, a is randomly chosen vector, for projecting the secret key s, and e is a random noise component, added to m+a.Math.s.

(7) To decrypt the message, someone possessing the secret key s may compute the quantity b−a.Math.s (equal to m+e), and then round the result to remove the noise component e and retrieve the message m. Of course, the noise term e has to be, and to remain small enough, if one wants to be able to retrieve the message m.

(8) When two encrypted messages are summed up, or even worse, when they are homomorphically multiplied together, one obtains an encrypted message, which is an encrypted version of the sum, or product of the two initial, unencrypted messages, whose noise component is higher than for the two initial encrypted messages.

(9) So, to prevent the noise term from increasing and increasing in the course of data processing, a refreshing procedure, usually named “bootstrapping”, is executed repeatedly, usually after each operation on the encrypted message c. This procedure produces a refreshed version of c, that is an encrypted message c′ which is decrypted as m (when decrypted using s), and whose noise component is smaller than the one of c.

(10) The bootstrapping procedure usually comprises the homomorphic (that is: in encrypted form) computation of X.sup.q.Math.V(X) where: V(X) is the following polynomial: 1+X+X.sup.2+X.sup.3+ . . . +X.sup.N−1, and q is equal to the integer number that is the nearest to 2N(b−a.Math.s): $q = \lfloor 2N(b - a.Math.s) \rceil$

(11) The constant term of X.sup.q.Math.V(X) mod (X.sup.N+1), that is coef.sub.0(X.sup.q.Math.V(X)), is a refreshed version of c: it is an encrypted version of m, but with a noise component e′ smaller than the noise component e of c.

(12) One may note that the initial noise e is somehow washed out when (homomorphically) computing q, thanks to the rounding operation $\lfloor \rceil$. In this operation, the factor 2N plays the role of an expansion factor, that enables to spread the possible values of b−a.Math.s (impaired by some noise) sufficiently, before rounding.

(13) Such a bootstrapping procedure is very useful and ingenious, but also very time-consuming (in particular because the operations involved have to be carried on homomorphically), all the more that it has to be executed repeatedly.

(14) The homomorphic encryption scheme succinctly described above, which combines “learning with error” and a bootstrapping procedure, is usually carried on with binary messages (that is, messages whose values are either 0 or 1). Indeed, in this technical field, most of the tools have been specifically developed and optimized for such messages.

(15) To perform homomorphic operations on a longer message, the message is decomposed into binary messages (classical binary decomposition), each binary message is encrypted separately, and the operation is then carried on, homomorphically, bit by bit. But in this process, one has to take into account the (encrypted) carry resulting from each binary addition, and to propagate this carry to the next binary operation. The process is thus a serial one, which is time consuming. Besides, each binary operation is carried homomorphically, which multiplies the number of homomorphic (and thus time-consuming) operations.

SUMMARY

(16) In this context, a homomorphic encryption technology, that enables to process efficiently messages longer than binary ones, is disclosed.

(17) An aspect of this technology concerns a method for determining an encrypted message c.sub.i, which is an encrypted version of a message m.sub.i, c.sub.i being equal to (a.sub.i,b.sub.i) where b.sub.i is determined from m.sub.i and from a secret key s, b.sub.i being equal to m.sub.i+e.sub.i+a.sub.i.Math.s with a.sub.i being a randomly selected vector for projecting s and e.sub.i being a random noise component added to m.sub.i+a.sub.i.Math.s,

(18) wherein the message m.sub.i belongs to a discrete set consisting of p.sub.i distinct elements, with p.sub.i being an odd number and wherein the elements of said discrete set are such that twice the difference between any two of these elements is not an integer number, said discrete set being the discrete torus

(19) $T_{\mathrm{pi}} = \frac{1}{p_i}\{-\frac{p_i - 1}{2}, -\frac{p_i - 1}{2} + 1, \text{ .Math. }, \frac{p_i - 1}{2} - 1, \frac{p_i - 1}{2}\}$,

or the set T.sub.pi,N[X] of polynomials of degree N−1 whose coefficients belong to T.sub.pi, or another discrete set in bijective relationship with T.sub.pi.

(20) The inventors have developed a homomorphic cryptographic technique for such “long” messages (longer than binary messages), enabling, among other things, adding, multiplying and refreshing (i.e.: bootstrapping) these messages while they remain encrypted. These operations are carried on without decomposing the message into binary, independently encrypted, smaller messages. In other words, the encrypted message is treated as a whole, with no decomposition onto smaller fields.

(21) So, the burden of dealing with encrypted carries and serial processing of the numerous and encrypted bits is avoided.

(22) It may be noted that the fact that a homomorphic processing is possible for long messages (without decomposing it), and the fact that this

processing can be fast and efficient is far from being an immediate, straightforward result.
(23) Indeed, as mentioned above, most of the homomorphic cryptography tools currently available are adapted specifically to binary messages. So, the inventors had to go against the usual practice and prejudices and had to move forward in the development of specific (and elaborated) tools intended for longer messages, without knowing whether a direct homomorphic processing of long messages was actually possible or not, or whether it could be executed efficiently or not. In particular, the inventors had to elaborate a new homomorphic multiplication scheme and a new bootstrapping method (presented below), to develop such an encryption protocol. Only once these quite complex tools had been developed could the inventors be sure that such a treatment could actually be carried out.
(24) And, in the process of developing these new tools, the inventors realized that working with messages belonging to the discrete Torus T.sub.pi, or to the set of polynomials T.sub.pi,N[X], or to another equivalent discrete set, with: pi being an odd number, and such that for any couple grouping two of the elements of said set, twice the difference between these two elements is not an integer number
was very fruitful, as it turns out that these specific features make different operations, involved in the multiplication and bootstrapping process mentioned above, feasible (as will be described in more details later).
(25) The fact that a direct homomorphic processing of a “long” message is not only possible, but that it can also be achieved efficiently, in a limited time, is also far from obvious, at first glance.
(26) Indeed, when the message m.sub.i is long, that is when the integer number p.sub.i is quite large, for example equal to 17, 21, or more, one would expect at first sight to have to use polynomials whose degree N−1 is high, during bootstrapping operations. Indeed, as explained above, in the calculation of the quantity q, the integer N acts as a kind of expansion coefficient. So, when p.sub.i is large, as the possible values of the message m.sub.i (distributed in the interval]−1/2;1/2]) are then close to each other, one expects to have to choose a number N that is large, in order to be able to spread these values sufficiently apart, before the rounding operation involved in the calculation of the quantity q. And a large N considerably increases the calculation time.
(27) More precisely, a rough estimate leads to the conclusion that N would have to be of the order of 10000 (with pi=20 and n=500, for instance), or greater, for the bootstrapping operation to work properly (that is with a low probability of losing of degrading the message m during this operation). And such values of N are almost prohibitive, in terms of computation time.
(28) But in fact, a complete, acute probability calculation shows that such a bootstrapping operation can be successfully performed with polynomials whose degree N−1 is much smaller than what the first approximate estimate mentioned above suggests. More precisely, it can be shown that an integer number N of the order of 1000 (e.g. 1024) turns out to be sufficient for typical values of pi (about 20-40, for example).
(29) According to the disclosed technology, N may be selected as small as 50 times p.sub.i, or below, or even below 30 times p.sub.i, to take advantage of this surprisingly favorable scaling and accelerate the bootstrapping processing.
(30) The method for bootstrapping developed by the inventors, mentioned above, is a method for bootstrapping an encrypted message c.sub.i= (a.sub.i,b.sub.i) determined as described above, this method comprising a determination of a refreshed encrypted message c.sub.i′=(a.sub.i′,b.sub.i′), which is an encrypted version of a value g(m.sub.i) of a function g applied to the message m.sub.i, c.sub.i′ having a noise component e.sub.i′ smaller than e.sub.i.
(31) The determination of c.sub.i′ comprises a step of determining homomorphically the constant coefficient coef.sub.0(X.sup.q.Math.W.sub.g(X)) of X.sup.q.Math.W.sub.g(X) mod (X.sup.N+1), where W.sub.g(X)=Σ.sub.j=0.sup.N−1w.sub.j.Math.X.sup.j is a polynomial of degree N−1, whose coefficients w.sub.j are given by the following formula:
(−1).sup. ⌈ ⌊2Nm.sup.i.sup.⌉ /N⌉ .*Math.w*.sub.j=*g*(*m*.sub.i), where $j=N\,⌈\,⌊2Nm.\mathrm{sub}.i⌉\,/N⌉ - ⌊2Nm.\mathrm{sub}.i⌉$
with ⌊2Nm.sub.i⌉ being the integer number that is the nearest to 2Nm.sub.i and with ⌈ ⌊2Nm.sub.i⌉ /N⌉ being the ceiling function applied to ⌊2Nm.sub.i⌉ /N, and where q is the integer number that is the nearest to 2N(b.sub.i−a.sub.i.Math.s): q= ⌊2N (b.sub.i−a.sub.i.Math.s)⌉ .
(32) So, during this bootstrapping operation, in addition to reducing (refreshing) the noise component of a cipher text, an arbitrary function g can be applied to the corresponding message m.sub.i.
(33) This accelerates considerably numerous data processing routines. Indeed, to apply such a function g to a message, instead of: carrying on an elementary operation (homomorphically), then bootstrapping the result (to counter the noise increase due to the preceding operation), carrying on another elementary operation, then bootstrapping the result, and so on, one just has to execute one time the special bootstrapping procedure presented above.
(34) In this bootstrapping procedure, the degree N−1 of the polynomial Wg(X) may be selected so as to be above p.sub.i, or even above 5 times p.sub.i.
(35) The disclosed technology concerns also a method for homomorphically multiplying two messages m**1**.sub.i and m**2**.sub.i, each belonging to the discrete torus T.sub.pi or to the set of polynomials T.sub.pi,N[X], the method comprising a determination of an encrypted message c**3**.sub.i which is an encrypted version of the product of m**1**.sub.i by m**2**.sub.i, c**3**.sub.i being determined from encrypted versions c**1**.sub.i and c**2**.sub.i of the messages m**1**.sub.i and m**2**.sub.i, without decrypting c**1**.sub.i or c**2**.sub.i, the method comprising homomorphically determining
(36) $\frac{(p_i+1)}{2}$ .Math. $[\frac{(p_i+1)}{2}$ .Math. p_i .Math. $(m1_i + m2_i)^2$ - $\frac{(p_i+1)}{2}$ .Math. p_i .Math. $(m1_i - m2_i)^2]$
Where p.sub.i.Math.(m**1**.sub.i+m**2**.sub.i).sup.2 and p.sub.i.Math.(m**1**.sub.i−m**2**.sub.i).sup.2, or
(37) $\frac{(p_i+1)}{2}$ .Math. p_i .Math. $(m1_i + m2_i)^2$ and $\frac{(p_i+1)}{2}$ .Math. p_i .Math. $(m1_i - m2_i)^2$, or $[\frac{(p_i+1)}{2}]^2$ .Math. p_i .Math. $(m1_i + m2_i)^2$ and $[\frac{(p_i+1)}{2}]^2$ .Math. p_i .Math. $(m1_i - m2_i)^2$
are determined homomorphically by executing the method for bootstrapping presented above, the function g being the function g.sub.sq′:m.sub.i.fwdarw.p.sub.i.Math.(m.sub.i).sup.2, or
(38) $_{sq}: m_i$ .fwdarw. $\frac{(p_i+1)}{2}$ .Math. p_i .Math. $(m_i)^2$,
or, respectively,
(39) $_{sq''}: m_i$ .fwdarw. $[\frac{(p_i+1)}{2}]^2$ .Math. p_i .Math. $(m_i)^2$.
(40) It may be noted that the tools presented above, for homomorphic processing of messages m.sub.i belonging to such a discrete set of p.sub.i elements (with p.sub.i odd), can, in addition to their own benefits, be applied in an extremely beneficial manner to a cryptographic technic for even longer messages, decomposed into smaller components m.sub.i, based on the Chinese remainder theorem decomposition, which, again, enables to avoid the burden of dealing with carries (so, in other words, the tools mentioned above are some kinds of basic bricks of this Chinese-remainder processing method, presented in more detail below).
(41) The disclosed technology concerns also a method for determining an encrypted message c.sub.f,i, which is an encrypted version of a value f(m**1**.sub.i,m**2**.sub.i) of a two-variable function f applied to messages m**1**.sub.i and m**2**.sub.i, m**1**.sub.i and m**2**.sub.i belonging each to the discrete set mentioned above (for instance the torus T.sub.pi or the set of polynomials T.sub.pi,N[X]), c.sub.f,i being determined from encrypted messages c**1**.sub.i and c**2**.sub.i without decrypting c**1**.sub.i or c**2**.sub.i, c**1**.sub.i and c**2**.sub.i being encrypted versions of m**1**.sub.i and m**2**.sub.i respectively, that have been determined according to the method for encrypting presented above, the determination of c.sub.f,i comprising: determining c**1′**.sub.i and c**2′**.sub.i, from c**1**.sub.i and c**2**.sub.i respectively, by executing the method for bootstrapping presented above, the function g employed during this bootstrapping being a function g.sub.map that maps said discrete set onto a set S whose elements μ.sub.j belong each to the torus T=]−1/2,1/2] and are such that, for any set of indexes (i,j,k,l) with (i,j)≠(k,l), (k,l) being different from (j,i): α.Math.μ.sub.i+β.Math.μ.sub.j is different from α,μ.sub.k+β.Math.μ.sub.l modulo ½, with α and β being two constant and integer, given numbers, c**1′**.sub.i and c**2′**.sub.i being the encrypted versions

of m**1**.sub.i=g.sub.map(m**1**.sub.i) and m**2**.sub.i=g.sub.map(m**2**.sub.i) respectively, where m**1**.sub.i and m**2**.sub.i belong each to S, determining homomorphicically, from c**1**′.sub.i and c**2**′.sub.l, an encrypted message c.sub.s which is an encrypted version of α.Math.m**1**.sub.i+β.Math.m**2**.sub.i, determining c.sub.f,i from c.sub.s by executing the method for bootstrapping presented above, the function g employed during this bootstrapping being a one-variable function g.sub.f from T to T.sub.pi, defined by the following condition: for any couple of messages m**1**.sub.j, m**2**.sub.j belonging each to said discrete set,

g.sub.f(α.Math.*m*1.sub.j+β.Math.*m*2.sub.j)=*f*(*m*1.sub.j, *m*2.sub.j) that is:

g.sub.f*[α.Math.g*.sub.map(*m*1.sub.j)+β.Math.*g*.sub.map(*m*2.sub.j)]=*f*(*m*1.sub.j, *m*2.sub.j).

(42) According to an aspect of this technology, the elements of the set S are such that for any set of indexes (i,j,k,l) with (i,j)≠(k,l), α.Math.μ.sub.i+β.Math.μ.sub.j is different from α.Math.μ.sub.k+β.Math.μ.sub.l modulo ½, even when (k,l)=(j,i). In this case, one may have, in particular: α=1 and β=−1, c.sub.s being determined by computing the homomorphic difference ⊖ between c**1**′.sub.i and c**2**′: c.sub.s=c**1**′.sub.i⊖c**2**′.sub.i, or α=−1 and β=1, c.sub.s being determined by computing the homomorphic difference ⊖ between c**2**′.sub.i and c**1**′: c.sub.s=c**2**′.sub.i⊖c**1**′.sub.i.

(43) According to another aspect of this technology, the function f is symmetric, f(m**1**.sub.i,m**2**.sub.i) being equal to f(m**2**.sub.i,m**1**.sub.i) for any couple of messages m**1**.sub.i and m**2**.sub.i, α and β are each equal to 1, and c.sub.s is determined by computing the homomorphic sum ⊕ of c**1**′.sub.i and c**2**′: c.sub.s=c**1**′.sub.i⊕c**2**′.sub.i.

(44) The disclosed technology also concerns such a method for determining an encrypted message c.sub.f,i, which is an encrypted version of a value f(m**1**.sub.i,m**2**.sub.i) of a two-variable function f, as such, independently from the bootstrapping method in question. Indeed, as the skilled person will appreciate, the method for homomorphically evaluating f, based on the intermediate mapping of said discrete set (e.g.: T.sub.pi) onto S, can be achieved independently from this specific bootstrapping procedure, using any method for homomorphically mapping said discrete set on S (the mapping method being possibly different from the one described above), and using then any method for homomorphically applying a function like the function g.sub.f to the quantity α.Math.m**1**.sub.i+β.Math.m**2**.sub.i.

(45) According to an aspect of the disclosed technology, any of the methods presented above is executed by a computer (programmed or otherwise arranged to execute the method in question), that is an electronic device or system (possibly distributed among several distant, remote devices) comprising at least one processor for executing logical operations, and one memory device for storing data.

(46) In particular, an aspect of the technology concerns an encrypting device, comprising one or more processors and at least one memory, the encrypting device being programmed to make the processor or processors executing the following steps: receiving data representative of an unencrypted message m.sub.i, that belongs to the discrete set mentioned above, processing said data according to the method for encrypting presented above, to determine encrypted data representative of the encrypted message c.sub.i, the secret key s employed during said processing, stored at least transitorily in the memory of the encrypting device, being accessed by the processor or processors during said processing.

(47) An aspect of the disclosed technology concerns a cryptographic system comprising such an encrypting device and a processing device, the processing device comprising one or more processors and at least one memory, the processing device being programmed to make the processor or processors executing the following steps: receiving data, representative of an encrypted message c.sub.i produced by the encrypting device, processing said data, according to the method for bootstrapping that has been presented above, to determine refreshed data, representative of the encrypted message c′.sub.i, said processing being achieved without decrypting c.sub.i and without using, reading or accessing the secret key s.

(48) In particular, the secret key s may be completely absent from the processing device (which is a separate device distinct from the encrypting device), and remain absent from the processing device at all time.

(49) The disclosed technology also concerns such a processing device, as such, independently from the encrypting device.

(50) The processing device may also be programmed to execute the method for multiplying presented above, and the method for determining an encrypted message c.sub.f,i, which is an encrypted version of a value f(m**1**.sub.i,m**2**.sub.i) of the two-variable function f.

(51) The disclosed technology concerns also a computer-readable program product comprising instructions that, when executed by a computer, make the computer to execute any of the method presented above.

(52) The disclosed technology concerns also a method for determining an encrypted message C, which is an encrypted version of a message x belonging to custom character.sub.p=custom character/pcustom character, p being equal to Π.sub.i=1.sup.rp.sub.i where the integer numbers p.sub.i are pairwise coprime, the method comprising: decomposing x into its components x.sub.i, i=1 . . . r, with x.sub.i equal to x modulo p.sub.i, for each component x.sub.i whose associated factor p.sub.i is odd, determining an encrypted version c.sub.i of a message m.sub.i, according to the method for encrypting that has been presented above m.sub.i being, among the elements of said discrete set, the element that is associated to x.sub.i by a given bijective relationship between custom character.sub.pi and said discrete set, if a component x.sub.i is associated to a factor p.sub.i equal to 2, determining an encrypted version c.sub.i of a quantity Q.sub.i associated to x.sub.i and that can takes two distinct values c.sub.i being equal to (a.sub.i,b.sub.i) with b.sub.i=Q.sub.i+e.sub.i+a.sub.i.Math.s, returning C=(c.sub.1, . . . c.sub.i, . . . c.sub.r).

(53) An aspect of this technology concerns a method for bootstrapping the encrypted message C=(c.sub.1, . . . c.sub.i, . . . c.sub.r) determined according the encrypting method that has just been described, this (generalized) bootstrapping method comprising: for each c.sub.i whose associated factor p.sub.i is odd, determining a refreshed encrypted message C′.sub.i, from c.sub.i, by executing the method for bootstrapping that has been presented before, returning C′=(c′.sub.1, . . . , c′.sub.i, . . . c′.sub.r).

(54) This technology concerns also a method for determining an encrypted message C**3**=(*c***3**.sub.1, . . . c**3**.sub.i, . . . c**3***r*), which is an encrypted version of the product of two messages x**1** and x**2**, each belonging to custom character.sub.p, C**3** being determined from encrypted messages C**1** and C**2**, without decrypting C**1** or C**2**, C**1** and C**2** being encrypted versions of x**1** and x**2** respectively, C**1**=(*c***1**.sub.1, . . . c**1**.sub.i, . . . c**1**.sub.r) and C**2**=(*c***2**.sub.1, . . . c**2**.sub.i, . . . c**2**.sub.r) having been determined from x**1** and x**2** respectively, according to the method for encrypting that has just been presented, wherein said discrete set is T.sub.pi or T.sub.pi,N[X], and wherein each component c**3**.sub.i of C**3** is determined from the corresponding components c**1**.sub.i and c**2**.sub.i of C**1** and C**2**, according to the method for multiplying that was described above for messages belonging to the discrete torus T.sub.pi, when c**1**.sub.i and c**2**.sub.i are associated to a factor p.sub.i that is odd.

(55) The disclosed technology concerns such a method for bootstrapping a message C, or such a method for multiplying homomorphically two messages x**1** and x**2**, in itself, independently from the bootstrapping method (for messages belonging to T.sub.pi) that has been presented above. Indeed, as the skilled person will appreciate, these two methods, for processing encrypted messages, associated to unencrypted messages x belonging to Z.sub.p (with p being equal to Π.sub.i=1.sup.rp.sub.i where the integer numbers p.sub.i are pairwise coprime), take advantage of the Chinese remainder theorem to process the encrypted “sub-messages” c.sub.i, i=1 . . . r, independently from each other, without having to deal with, or anyhow considering (encrypted) carries. This technique could clearly be applied using other “sub-routines” than the one presented above, for the homomorphic processing of the sub-messages m.sub.i. In particular, any multiplying or bootstrapping method for elements belonging to the discrete set mentioned above (e.g.: the discrete torus T.sub.pi) could be used, instead of the one described above.

(56) According to the disclosed technology, these encrypting and processing techniques (based on a “Chinese remainder” decomposition), could be executed by a computer.

(57) The disclosed technology concerns also an electronic device, comprising at least one processor and one memory, programmed or otherwise configured to execute one of these methods. The disclosed technology concerns also a computer-readable program product comprising instructions, whose execution by a computer make the computer to execute anyone of these methods.

Description

BRIEF DESCRIPTION OF THE DRAWINGS

(1) Other characteristics and benefits of the disclosed technology will become clear from the description which is given below, by way of example and non-restrictively, in reference to the figure, in which:

(2) FIG. **1** represents, as a block-diagram, operations carried on to multiply homomorphically two messages.

(3) FIG. **2** represents, as a block-diagram, operations carried on to evaluate homomorphically a two-variable function.

(4) FIG. **3** represents, as a block-diagram, an encryption step of a long message x, based on a Chinese remainder decomposition.

(5) FIG. **4** represents, as a block-diagram, operations carried on to add homomorphically two encrypted messages obtained previously according to the encryption step of FIG. **3**.

(6) FIG. **5** represents, as a block-diagram, operations carried on to multiply homomorphically two encrypted messages, obtained previously according to the encryption step of FIG. **3**, said multiplication comprising several sub-multiplications carried on according to the process represented in FIG. **1**.

(7) FIG. **6** schematically illustrate a cryptographic system according to the disclosed technology.

DETAILED DESCRIPTION

(8) As mentioned above, the disclosed technology concerns an homomorphic encryption technique that enables to carry on operations on encrypted messages without decrypting them, these encrypted messages being associated to unencrypted messages that are each longer than a single bit, and that are processed without being decomposed into individual binary messages.

(9) The disclosed technology concerns more particularly a technique for encrypting and processing messages m.sub.i, that belong to a discrete set consisting of p.sub.i distinct elements, with p.sub.i being an odd number, these messages being encrypted and processed directly, without decomposing these messages into smaller (for instance binary) messages that would be encrypted separately. This technology is presented below in the case where the discrete set in question is the discrete torus

(10) $T_{\mathrm{pi}} = \frac{1}{p_i}\{-\frac{p_i - 1}{2}, -\frac{p_i - 1}{2} + 1,$ .Math. $, \frac{p_i - 1}{2} - 1, \frac{p_i - 1}{2}\}$,

or the set T.sub.pi,N[X] of polynomials of degree N−1 whose coefficients belong to T.sub.pi. Still, as the skilled person will appreciate, this technology can be applied as well (possibly, with minor modifications for the homomorphic addition or multiplication) to another discrete set of messages in bijective relationship with T.sub.pi, like

(11) $\mathbb{Z}_{\mathrm{pi}} = \mathbb{Z} / p_i\mathbb{Z}or\frac{1}{p_i}\{-\frac{p_i - 1}{2} + 1,$ .Math. $, \frac{p_i - 1}{2} - 1, \frac{p_i - 1}{2}, \frac{p_i - 1}{2} + 1\}$

(that is T.sub.pi with an offset of +1/p.sub.i; it may be noted that for this alternative example, the different operations described below—including the bootstrapping procedure, but also the homomorphic multiplication—can be implemented in the same manner as in the case of T.sub.pi, even if the values may have to be shifted to reintegrate them in the interval [−1/2, 1/2[) for instance.

(12) The disclosed technology concerns also a technique for encrypting and processing even longer messages x, using a specific decomposition of the message x into smaller messages m.sub.i, this decomposition being based on the Chinese remainder theorem decomposition and enabling to process the sub-messages m.sub.i separately from each other, without having to take into account possible carries.

(13) The encryption technique for encrypting and processing a message m.sub.i, without decomposing it, will be presented first. The technique for encrypting and processing a longer message x, composed of several such messages m.sub.i, based on a Chinese remainder theorem decomposition, will be presented then. Finally, electronic devices and systems for implementing such methods will be presented.

(14) Homomorphic encryption and processing of a message m.sub.i belonging to T.sub.pi, or to T.sub.pi,N[X].

(15) As mentioned above, this homomorphic encryption technique is applied to messages m.sub.i that belong to the discrete torus

(16) $T_{\mathrm{pi}} = \frac{1}{p_i}\{-\frac{p_i - 1}{2}, -\frac{p_i - 1}{2} + 1,$ .Math. $, \frac{p_i - 1}{2} - \frac{p_i - 1}{2}\}$,

or to the set T.sub.pi,N[X] of polynomials of degree N−1 whose coefficients belong to T.sub.pi.

(17) More precisely, the set of polynomials T.sub.pi,N[X] is the finite field composed of the polynomials of degree N−1 defined modulo X.sup.N+1, and whose coefficients belong to T.sub.pi: T.sub.pi,N[X]=T.sub.pi[X]/(X.sup.N+1).

(18) Remarkably, the integer number p.sub.i is odd (which makes some of the operations described below possible).

Encryption

(19) An unencrypted message m.sub.i, belonging to T.sub.pi, is encrypted according to the “learning with error” scheme: the encrypted message c.sub.i=(a.sub.i,b.sub.i) (or, in other words, the cypher text c.sub.i), which is the encrypted version of m.sub.i, is determined by: randomly choosing a vector a.sub.i, and computing the quantity b.sub.i=m.sub.i+e.sub.i+a.sub.i.Math.s where s is a secret key and where e.sub.i is a noise component, added to m.sub.i+a.sub.i.Math.s.

(20) When m.sub.i belongs to T.sub.pi, the noise component e.sub.i belongs to the torus T=]−1/2,1/2]. And when m.sub.i belongs to T.sub.pi,N[X], e.sub.i belongs to the set T.sub.N[X] of polynomials of degree N−1, defined modulo (X.sup.N+1) and whose coefficients belong each to T. e.sub.i is randomly chosen according to a predefined statistical distribution. At least one feature of this distribution, for instance its standard deviation (or the standard deviation of each of its coefficients, when e.sub.i is a polynomial), is determined depending on the integer number p.sub.i. For instance, the standard deviation of this distribution is all the smaller than p.sub.i is high (this standard deviation may be inversely proportional to p.sub.i, for example).

(21) The secret key s is a vector having n components, each belonging to the set of integers Z, or to the set of polynomials of degree N−1, defined modulo (X.sup.N+1) and whose coefficients belong each to Z. In other words, s belongs to Z.sup.n, or to Z.sub.N[X].sup.n, where Z.sub.N[X]=Z[X]/(X.sup.N+1). The secret key s may more particularly belong to B.sup.n, or to B.sub.N[X]n, where B={0, 1} (B being the set constituted by the binary values 0 and 1).

(22) The vector a.sub.i has also n components, and is such that the scalar product a.sub.i.Math.s belongs either to T.sub.pi (when m.sub.i belongs to T.sub.pi), or to T.sub.pi,N[X] (when m.sub.i belongs to T.sub.pi,N[X]). For instance, if m.sub.i belongs of T.sub.pi, and if s belongs to B.sup.n, a.sub.i belongs to T.sub.pi.sup.n (each of the n components of a.sub.i belongs to T.sub.pi).

(23) In the following, for the sake of clarity, we consider that m.sub.i belongs to T.sub.pi, and that s belongs to Z.sup.n or B.sup.n.

(24) Besides, in the following, we consider more specifically that s belongs to B.sup.n (which simplifies some of the operations involved in the bootstrapping procedure described below).

(25) To decrypt the encrypted message c.sub.i=(a.sub.i,b.sub.i), a decryption module or a decryption device that has access to the secret key s computes b.sub.i−a.sub.i.Math.s (which is equal to m.sub.i+e.sub.i), and then rounds it to the nearest value belonging to the discrete Torus T.sub.pi, thus removing the noise component e.sub.i and obtaining m.sub.i.

(26) The encrypted message c.sub.i, obtained by encrypted m.sub.i using the secret key s, may be also noted {tilde over (φ)}.sub.s(m.sub.i).

Homomorphic Operations

(27) An objective of the instant technology is to carry on operations homomorphically, that is to say to carry on operations on one, two or more messages m**1**.sub.i, m**2**.sub.i, . . . , but using their encrypted version c**1**.sub.i, c**2**.sub.i, . . . (instead of the unencrypted messages m**1**.sub.i, m**2**.sub.i, . . .) and without decrypting them, without knowing or having access to the secret key s.

(28) More generally, determining homomorphically a given quantity is meant determining an encrypted version of this quantity, on the basis of

encrypted messages, without using or determining unencrypted versions (unencrypted counterparts) of these encrypted messages, and without using (in particular without accessing) the secret key s.
(29) In practice, it is particularly interesting to be able to achieve two-operands homomorphic operations, like a homomorphic addition, multiplication or comparison of two messages (which is very useful for sorting encrypted data without decrypting them).
(30) In the case of a two-operands operation, the usual ("normal") operation, whose operands are m**1**.sub.i and m**2**.sub.i, is noted Op and its result is noted m**3**.sub.i: m**3**.sub.i=Op(m**1**.sub.i,m**2**.sub.i). The corresponding homomorphic operation (that is, the homomorphic counterpart of Op) is noted Hop. So, the homomorphic operation Hop is an operation: whose operands are two cipher texts c**1**.sub.i and c**2**.sub.i, which are encrypted versions of two messages m**1**.sub.i and m**2**.sub.i respectively (each belonging to T.sub.pi); which produces a cipher text c**3**.sub.i=Hop(c**1**.sub.i,c**2**.sub.i), that it determined from c**1***i* and c**2***i* without decrypting c**1***i* or c**2**.sub.i, without using the secret key s (in particular, without knowing or accessing the secret key s); the cipher text c**3**.sub.i being an encrypted version of m**3**.sub.i=Op(m**1**.sub.i,m**2**.sub.i).
(31) In other words, when decrypting c**1**.sub.i, c**2**.sub.i and c**3**.sub.i using the secret key s, one gets respectively m**1**.sub.i, m**2**.sub.i and m**3**.sub.i=Op(m**1**.sub.i,m**2**.sub.i).
Homomorphic Addition
(32) The homomorphic addition is noted ⊕, (Hop=⊕). The homomorphic addition of two cipher texts, c**3**.sub.i=c**1**.sub.i⊖c**2**.sub.i, is readily achieved by summing the two cipher texts c**1**.sub.i=(a**1**.sub.i,b**1**.sub.i) and c**2**.sub.i=(a**2**.sub.i,b**2**.sub.i) component by component: c**3**.sub.i=(a**3**.sub.i,b**3**.sub.i) with a**3**.sub.i=a**1**.sub.i+a**2**.sub.i and b**3**.sub.i=b**1**.sub.i+b**2**.sub.i.
(33) Indeed, b**3**.sub.i−s.Math.a**3**.sub.i=m**1**.sub.i+m**2**.sub.i+e**1**.sub.i+e**2**.sub.i is rounded as m**1**.sub.i+m**2**.sub.i (provided that e**1**.sub.i+e**2**.sub.i is small enough). So, c**3**.sub.i=c**1**.sub.i⊕c**2**.sub.i is indeed decrypted as m**3***i*=m**1**.sub.i+m**2**.sub.i.
(34) A homomorphic subtraction ⊖ can be achieved similarly by subtracting two cipher texts component by component.
(35) A homomorphic multiplication for messages belonging to the discrete torus is much more complicated to develop and to implement. It will be presented later, after having described a new bootstrapping technique for messages belonging to the discrete torus (as this bootstrapping technique is used to carry on the homomorphic multiplication in question).
Method for Bootstrapping an Encrypted Message c.SUB.i
(36) A method for bootstrapping an encrypted message c.sub.i=(a.sub.i,b.sub.i), that had been determined, as described above, by encrypting a message m.sub.i using the secret key s, is presented below.
(37) This method comprises a homomorphic determination of a refreshed encrypted message c.sub.i′=(a.sub.i′,b.sub.i′), which is an encrypted version of a value g(m.sub.i) of a function g applied to the message m.sub.i, c.sub.i′ having a noise component e.sub.i′ smaller than the noise component e.sub.i of c.sub.i. The result c.sub.i′ of this bootstrapping method, applied to c.sub.i, is noted G(c.sub.i).
(38) So, this new bootstrapping method is a bootstrapping method adapted to messages m.sub.i that belong to the discrete torus T.sub.pi. It enables to refresh encrypted messages c.sub.i associated to such messages m.sub.i, that is to say to reduce the noise components e.sub.i of these cypher texts c.sub.i. And it enables also, at the same time, to apply an arbitrary function g to the message m.sub.i. In other words, this method enables to homomorphically determine the value g(m.sub.i) of the function g, applied to m.sub.i.
(39) The function g can be any function from T.sub.pi to T.sub.pi. In other words, g maps elements of T.sub.pi to elements of T.sub.pi (or elements of T.sub.pi,NN[X] to elements of T.sub.pi,N[X]).
(40) The determination of c.sub.i′ comprises a step of determining homomorphically the constant coefficient coef.sub.0(X.sup.q.Math.W.sub.g(X)) of a polynomial X.sup.q.Math.W.sub.g(X) mod (X.sup.N+1), where q is the integer number that is the nearest to 2N(b.sub.i−a.sub.i.Math.s): q= ⌊2N(b.sub.i−a.sub.i.Math.s)┐ .
(41) The homomorphic determination of coef.sub.0(X.sup.q.Math.W.sub.g(X)) is carried on in a way similar to prior art techniques, like described in section 6 of Ilaria Chillotti, Nicolas Gama, Mariya Georgieva, and Malika Izabachène, "TFHE: Fast fully homomorphic encryption over the torus", Journal of Cryptology, 33(1):34-91, 2020, for instance, but using a specially designed polynomial W.sub.g(X), associated to the function g considered.
(42) The coefficients of the polynomial W.sub.g(X), of degree N−1, are noted w.sub.j:
w.sub.g(*X*)=Σ.sub.j=0.sup.N−1*w*.sub.j.*Math.X*.sup.j
(43) They belong each to T.sub.pi and are determined as follow.
(44) Let's consider a function f.sub.w, from Z to T.sub.pi, defined as:
f.sub.w:*z.fwdarw.f*.sub.w(*z*)=(−1).sup. ⌈z/N┐ .*Math.w*.sub.j′ where *j′=N* ⌈z/*N*┐ −z
⌈.Math.┐ denoting the ceiling function.
(45) The coefficients w.sub.j of W.sub.g(X) are then defined, depending on g, such that, for any m.sub.i belonging to T.sub.pi:
f.sub.w(⌊2*Nm*.sub.i┐)=*g*(*m*.sub.i)
(46) So, the coefficients w.sub.j are related to the function g by the following equation:
(−1).sup. ⌈ ⌊2Nm.sup.i.sup.┐ /N┐ .*Math.w*.sub.j′=*g*(*m*.sub.i), where *j′=N* ⌈ ⌊2*Nm*.sub.i┐ /*N*┐ − ⌊2*Nm*.sub.i┐
(47) It can be proven that, given that p.sub.i is odd, and that twice the difference between any two of the elements of the discrete set considered (namely T.sub.pi, here) is not an integer number: a) it is possible, for any function g from T.sub.pi to T (and more generally for any function from any discrete set of p.sub.i elements such as defined above, to T), to construct a corresponding function f.sub.w, and thus a polynomial W.sub.g(X), that fulfills the conditions given above, and that b) thanks to this particular choice of polynomial (i.e.: thanks to the use of the particular polynomial W.sub.g(X)), homomorphically determining coef.sub.0(X.sup.q.Math.W.sub.g(X)) actually leads to the determination of an encrypted version of g(m.sub.i).
(48) The detailed (and quite elaborate) proof of these results is not presented here, for the sake of conciseness and as it not directly useful in view of implementing the instant technology.
(49) Still, it is noted that point a) is far from being immediate. Indeed, it can be readily noted that, for any z belonging to Z,
f.sub.w(*z*+*N*)=−*f*.sub.w(*z*)=*f*.sub.w(*z*−*N*)
(50) So the function f.sub.w defined above is quite constrained, and a detailed analysis is in fact required to ensure that an adequate f.sub.w function can actually be constructed for any function g (provided that the conditions above, regarding p.sub.i and the elements of the set, are fulfilled).
(51) It may be noted that this bootstrapping technique, presented above in the case where the discrete set with p.sub.i elements is T.sub.pi (or T.sub.pi,NN[X]), can be applied as well in the case of another discrete set consisting of p.sub.i elements (with pi odd, and such that twice the difference between any two of the elements of this set is not an integer number). In particular, the formula giving the values of the coefficients w.sub.j of the polynomial W.sub.g remains the same.
(52) As mentioned in the section entitled "summary", the degree N−1 of the polynomial W.sub.g(X) can be chosen so that N is below 50 times p.sub.i, or even below 30 times p.sub.i, which is high enough to avoid enables erasing or degrading the message m.sub.i, while leading to a reasonably fast execution of this bootstrapping procedure.
(53) Besides, N may be chosen as equal to or higher p.sub.i, or even than 5 times p.sub.i. The coefficients w.sub.j of W.sub.g(X) are then numerous enough to adjust the polynomial W.sub.g(X) to any, arbitrary function g, from T.sub.pi to T.sub.pi.
(54) Some details regarding the homomorphic evaluation of coef.sub.0(X.sup.q.Math.W.sub.g(X)) in itself are presented now. They are outlined briefly, as this evaluation is carried on in a way similar to prior art techniques (such as the one described in ***Ref1), apart from the specific choice of the polynomial W.sub.g(X).

(55) As known in the art, a polynomial such as X.sup.q.Math.W.sub.g(X) modulo (X.sup.N+1) can be decomposed into several successive modular products:
X.sup.q.*Math*.*W*.sub.g(*X*)=*X*.sup.−ā.sup.n.sup.s.sup.n.Math.(. . . (*X*.sup.−ā.sup.1.sup.s.sup.1.Math.(*X*.sup.−b.*Math*.*W*.sub.g(*X*))) . . .)
where b−Σ.sub.k=1.sup.nā.sub.kS.sub.k≈ ⌊2*N*(*b*.sub.i−*a*.sub.i.*Math*.*s*)⌉ =*q*.
(56) In the equation above, each product ".Math." is a modular product: the left term, for instance X.sup.−b, or X.sup.−ā.sup.1.sup.s.sup.1, belongs to Z.sub.N[X], while the right term, for instance W.sub.g(X), or (X.sup.−b.Math.W.sub.g(X)), belongs to T.sub.pi,N[X]. Such a product is completely similar to the product y.Math.m.sub.i of an integer number y with a number m.sub.i belonging to T.sub.pi, which is usually designated as a modular product (due to the modular structure of T.sub.pi, defined modulo 1).
(57) It is known that a modular product y.Math.m.sub.i, or X.sup.j.Math.W.sub.g(X), can be computed homomorphically in the form of a so-called external product X.sub.S′(y)custom character{tilde over (φ)}.sub.s(m.sub.i) where {tilde over (φ)}.sub.s(m.sub.i)=c.sub.i is the encrypted version of the message m.sub.i (c.sub.i being computed as explained above), and where X.sub.S′(z) is the so-called TGSW encrypted form of z (TGSW standing for "Gentry, Sahai, Waters" Torus encryption, the TGSW scheme having been suggested in the article "Homomorphic encryption from learning with errors: Conceptually-simpler, asymptotically-faster, attribute-based", by C. Gentry, A. Sahai, and B. Waters, InCrypto '13, 2013).
(58) Besides, as known in the art, a quantity like X.sub.S′(X.sup.−ā.sup.k.sup.s.sup.k)custom character{tilde over (φ)}.sub.s(m) can be computed without knowing the secret key component s.sub.k, k=1 . . . n (more generally, without knowing the secret key s).
(59) Indeed, as the secret key components s.sub.k, k=1 . . . n belong each to B={0, 1}, the quantity X.sup.−ā.sup.k.sup.s.sup.k is equal to (X.sup.−ā.sup.k−1)s.sub.k+1 which is encrypted as (X.sup.−ā.sup.k−1).Math.X.sub.S′(s.sub.k)+H.sub.B.sup.l.
(60) And so, an encrypted version of X.sup.q.Math.W.sub.g(X) modulo (X.sup.N+1) can be computed without knowing or having access to the secret key s as it can be computed using an encrypted versions X.sub.S′(s.sub.k) of the secret key components s.sub.k, sometimes designated as a Bootstrap Key BK.sub.k in the literature. Usually, the Bootstrap Key BK.sub.k=X.sub.S′(s.sub.k), which is an encrypted version of the secret key component s.sub.k, encrypted using another key s′, is a public key.
(61) The way to compute the TGSW encrypted version X.sub.S′(y) of an integer number y or, equivalently, to compute the TRGSW (GSW Torus 'Ring') encrypted version X.sub.S′,(P) of a polynomial P belonging to Z.sub.N[X], is recalled below. The way to compute the external product custom character is also recalled.
(62) X.sub.S′(y) is defined as being equal to Z+yH.sub.B.sup.l with:
(63) $Z = \begin{pmatrix} z_1 \\ \text{.Math.} \\ z_{l(n+1)} \end{pmatrix}$,
where the z.sub.j, j=1 . . . l(n+1) are l(n+1) distinct encrypted versions of 0, the null value 0 being encrypted with s′:z.sub.j={tilde over (φ)}.sub.S′(0), and where l is a given, fixed integer number; and H.sub.B.sup.l a matrix whose dimension is (n+1)×(n+1)×l and whose coefficients belong to the Torus T, H.sub.B.sup.l being equal to:
(64) $0H_B^l = \begin{pmatrix} h & 0 & \text{.Math.} \\ 0 & h & \ddots \\ \text{.Math.} & \ddots & \ddots \end{pmatrix} \text{with} h = \begin{pmatrix} 1/B \\ \text{.Math.} \\ 1/B^l \end{pmatrix}$,
B being an integer number higher than or equal to 2.
(65) The external product Ccustom characterc, between: an encrypted message c={tilde over (φ)}.sub.S′(m), c∈T.sup.n+1, corresponding to a message m belonging to the discrete torus T.sub.pi, more generally to the torus T, and an encrypted version C=X.sub.S′(y) of an integer number y, C∈custom character(T), is computed as explained below.
(66) The n+1 components c.sub.j, j=1 . . . n+1 of the encrypted message c={tilde over (φ)}.sub.S′(m) are each decomposed, using a kind of numeral system of base B (like the one used to compute X.sub.S′):
(67) $c_j \approx \text{.Math.}_{u=1}^{l} \frac{c_j^u}{B^u} \bmod(1), 0 \leq \text{.Math.}\ c_j^u\ \text{.Math.} \leq B - 1$
(68) Then, one computes
Ccustom character$c=\Sigma$.sub.B,l(c)C
where Σ.sub.B,l(c) C denotes the usual row-vector matrix product and where Σ.sub.B,l(c) is a matrix of dimension (n+1)×l whose expression is:
Σ.sub.B,l(c)=(σ.sub.B,l(*c*.sub.1), . . . , σ.sub.B,l(*c*.sub.j), . . . , σ.sub.B,l(*c*.sub.n+1))
with σ.sub.B,l(c.sub.j)=(c.sub.j.sup.1, . . . , c.sub.j.sup.u, . . . , c.sub.j.sup.l).sup.T.
(69) The result, Ccustom characterc, which belongs to T.sup.n+1, is an encrypted version of y.Math.m. In other words, decrypting this quantity using s gives y.Math.m.
(70) This technique can be applied to any message m belonging to the torus T. So, it can also be applied, like here, to messages belonging to the discrete torus T.sub.pi.
Homomorphic Multiplication
(71) First, it is noted that the usual, natural multiplication cannot be used as such for multiplying two messages belonging to the discrete torus T.sub.pi.
(72) For elements belonging to the discrete torus T.sub.pi, a multiplication, noted ×.sub.T, can however be defined, according to the following equation:
*m*3.sub.i=*m*1.sub.i×.sub.T*m*2.sub.i:=(*x*1.sub.i.*Math*.*x*2.sub.i mod(*p*.sub.i))/*p*.sub.i=(*m*1.sub.i.*Math*.*m*2.sub.i).Math.*p*.sub.i mod(1)
where m**1**.sub.i=x**1**.sub.i/p.sub.i and m**2**.sub.i=x**2**.sub.i/p.sub.i each belong each to T.sub.pi, x**1**.sub.i and x**2**.sub.i being integer numbers, and where ".Math." is the usual, natural multiplication.
(73) Clearly, the product m**3**.sub.i=m**1**.sub.i×.sub.Tm**2**.sub.i belongs also to T.sub.pi. Besides, this multiplication between elements of the discrete torus complies with the modular nature of T.sub.pi. Indeed, for any couple of messages m**1**.sub.i, m**2**.sub.i belonging to T.sub.pi.sup.2:
(*m*1.sub.i+1)×.sub.T*m*2.sub.i=((*x*1.sub.i+*p*.sub.i).Math.*x*2.sub.i mod(*p*.sub.i))/*p*.sub.i=(*x*1.sub.i.*Math*.*x*2.sub.i mod(*p*.sub.i))/*p*.sub.i
as x**2**.sub.i is an integer number, and so, (m**1**.sub.i+1)×.sub.Tm**2**.sub.i=m**1**.sub.i×.sub.Tm**2**.sub.i.
(74) The homomorphic multiplication procedure described here is noted ⊙.
(75) It produces a cyphertext c**3**.sub.i=c**1**.sub.i⊙c**2**.sub.i which is an encrypted version of m**3**.sub.i=m**1**.sub.i×.sub.Tm**2**.sub.i, c**1**.sub.i and c**2**.sub.i being encrypted versions of m**1**.sub.i and m**2**.sub.i respectively.
(76) To determine c**3**.sub.i, that is to say to determine homomorphically the quantity (x**1**.sub.i.Math.x**2**.sub.i)/p.sub.i mod (1), the bootstrapping procedure described above is exploited, as well as the fact that xy=((x+y).sup.2−(x−y).sup.2)/4.
(77) This identity is employed to switch from the two-operands multiplication operation xy to a single-operand squaring operation, this squaring operation being carried on homomorphically thanks to this specific bootstrapping procedure (thanks to an appropriate choice of the function g).
(78) More precisely, to determine homomorphically m**1**.sub.i×.sub.Tm**2**.sub.i=(x**1**.sub.i.Math.x**2**.sub.i)/p.sub.i mod(1), one determines homomorphically the quantity M below:
(79) $M = \frac{(p_i+1)}{2}\ \text{.Math.}\ [\frac{(p_i+1)}{2}\ \text{.Math.}\ p_i\ \text{.Math.}\ (m1_i + m2_i)^2 - \frac{(p_i+1)}{2}\ \text{.Math.}\ p_i\ \text{.Math.}\ (m1_i - m2_i)^2]$
where (m**1**.sub.i+m**2**.sub.i).sup.2=(m**1**.sub.i+m**2**.sub.i).Math.(m**1**.sub.i+m**2**.sub.i), ".Math." being the usual, natural multiplication.

(80) Indeed,

(81) $\frac{(p_i+1)}{2}$ .Math. [$\frac{(p_i+1)}{2}$ .Math. p_i .Math. $(m1_i + m2_i)^2$ - $\frac{(p_i+1)}{2}$ .Math. p_i .Math. $(m1_i - m2_i)^2$] = $\frac{(p_i+1)}{2}$ .Math. [$\frac{(p_i+1)}{2}$ .Math. $\frac{(x1_i + x2_i)^2}{p_i}$ - $\frac{(p_i+1)}{2}$ .Math. $\frac{(x1_i - x2_i}{p_i}$

as (pi+1)/2 is the inverse of 2, in Z.sub.pi.

(82) In this respect, it is noted that it is useful to have p.sub.i odd, as it allows 2 to be inversible, in Z.sub.pi.

(83) To determine homomorphically the quantity

(84) $\frac{(p_i+1)}{2}$ .Math. p_i .Math. $(m1_i + m2_i)^2$ - $\frac{(p_i+1)}{2}$ .Math. p_i .Math. $(m1_i - m2_i)^2$

one computes G.sub.sq(c**1**.sub.i⊕c**2**.sub.i)⊖G.sub.sq(c**1**.sub.i⊖c**2**.sub.i) where G.sub.sq denotes the bootstrapping procedure described above, in the particular case where the function g is the function:

(85) $g_{sq}: m_i = x_i / p_i$ .fwdarw. $\frac{(p_i+1)}{2}$ .Math. p_i .Math. $(m_i)^2 = \frac{(p_i+1)}{2}$ .Math. $[(x_i)^2 / p_i \bmod(1)]$

(86) And so, finally c**3**.sub.i is determined, homomorphically, from c**1**.sub.i and c**2**.sub.i, according to the following equation, where ⊙ custom character is the external product mentioned above (see FIG. **1**):

(87) $c3_i = c1_i \odot c2_i = {}_S(\frac{p_i+1}{2})[G_{sq}(c1_i \oplus c2_i) \ominus G_{sq}(c1_i \ominus c2_i)]$

(88) As the skilled person will appreciate, several modifications can be made to this homomorphic multiplication procedure, without departing from the disclosed technology. For instance, instead of using the bootstrapping "function" G.sub.sq to determine homomorphically the quantity M, one could use another bootstrapping "function", noted G.sub.sq, which corresponds to the bootstrapping procedure described above in the particular case where the function g is the function:

g.sub.sq′:*m*.sub.i=*x*.sub.i/*p*.sub.i.*fwdarw.m*.sub.i×.sub.T*m*.sub.i=*p*.sub.i.Math.(*m*.sub.i).sup.2=(*x*.sub.i).sup.2/*p*.sub.i mod(1)

(89) In this case the encrypted product ca is determined, homomorphically, according to the following equation:

(90) $c3_i = {}_S(\frac{p_i+1}{2})\ ({}_S(\frac{p_i+1}{2})G_{sq'}(c1_i \oplus c2_i)) \ominus ({}_S(\frac{p_i+1}{2})G_{sq'}(c1_i \ominus c2_i))]$

(91) Still, using G.sub.sq will enable a faster computation than G.sub.sq′, as it prevents from computing two extra external products ⊙ custom character.

(92) And for even faster computation, c**3**.sub.i could be determined homomorphically, from c**1**.sub.i and c**2**.sub.i, according to the following equation:

*c*3.sub.i=*c*1.sub.i⊙*c*2.sub.i=*G*.sub.sq″(*c*1.sub.i⊕*c*2.sub.i)⊖*G*.sub.sq″(*c*1.sub.i⊖*c*2.sub.i)

where G.sub.sq″ denotes the bootstrapping function presented above in the case where the function g is the function:

(93) $g_{sq''}: m_i = x_i / p_i$ .fwdarw. $[\frac{(p_i+1)}{2}]^2$ .Math. p_i .Math. $(m_i)^2 = [\frac{(p_i+1)}{2}]^2$ .Math. $[(x_i)^2 / p_i \bmod(1)]$

Homomorphic Evaluation of a Two-Variable Function

(94) The disclosed technology concerns also a method for evaluating a two-variable function, that is an arbitrary two-operands function, homomorphically.

(95) Being able to evaluate such a quantity is very useful for homomorphic data bases processing. Indeed, it enables, inter alia, to compare two ciphers texts c**1**.sub.i and c**2**.sub.i without decrypting them (for instance to sort the data base). To this end, the two-variable function employed could be the Max, or Min function. In this respect, one may note that a function returning (in an encrypted form) the maximum, or the minimum of a set of two messages is a two-variable function. And due to the modular nature of the messages employed here, the maximum or minimum of two messages cannot be determined by simply looking at the sign of the difference of these two messages.

(96) As explained below, to evaluate homomorphically a two-variable function f, each of the two inputs of this function (which belong each to T.sub.pi) is mapped on a specifically designed set of values S. The elements μ.sub.j of Sbelong each to T, and such that, for any set of indexes (i,j,k,l) with (i,j)≠(k,l, (k,l) being also different from (j,i): α.Math.μ.sub.i+β.Math.μ.sub.j is different from α.Math.μ.sub.k+β.Math.μ.sub.l, modulo ½, where α and β are two constant and integer, given numbers.

(97) So:

∀(*i*,*j*)≠(*k*,*l*), with (*j*,*i*)≠:α.Math.μ.sub.i+β.Math.μ.sub.j≠α.Math.μ.sub.k+β.Math.μ.sub.l and α.Math.μ.sub.i+β.Math.μ.sub.j≠α.Math.μ.sub.k+β.Math.μ.sub.l+1/2

(98) Then, taking advantage of this property, the two-operand function f is transformed into a one-operand function g.sub.f by first computing a linear combination of these two messages (multiplied respectively by the coefficients α and β), in order to obtain a single operand (the two messages having been preliminarily mapped onto S). The one-operand function g.sub.f is then homomorphically evaluated, by using the specific bootstrapping procedure described above.

(99) More precisely, the two input messages m**1**.sub.i and m**2**.sub.i are first mapped onto corresponding elements of S, thanks to a mapping function g.sub.map:

g.sub.map: ∀*m*∈*T*.sub.pi.*fwdarw.m*=*g*.sub.map(*m*)∈S

where m is one of the elements of S.

(100) g.sub.map is chosen among the different possible functions that map T.sub.pi on S.

(101) It may be noted that it can be proven that it is always possible to build a set S fulfilling the condition above (and fulfilling also one the additional conditions given below for S). In practice, the set S can be chosen among different sets fulfilling these conditions. Preferably, the set S that is chosen is the one for which the values μ.sub.j of the set are the more spaced apart from each other.

(102) The "mapped" messages associated respectively to m**1**.sub.i and m**2**.sub.i are noted m**1**.sub.i and m**2**.sub.i respectively:

m**1**.sub.i=g.sub.map(m**1**.sub.i), m**2**.sub.i=g.sub.map(m**2**.sub.i)

(103) The "one-operand" function g.sub.f corresponding to f is then defined as:

∀(*m*1.sub.j,*m*2.sub.j)∈*T*.sub.pi.sup.2*g*.sub.f(α.Math.*m*1.sub.j+β.Math.*m*2.sub.j)=*f*(*m*1.sub.j,*m*2.sub.j)

that is:

∀(*m*1.sub.j,*m*2.sub.j)∈*T*.sub.pi.sup.2*g*.sub.f[α.*Math*.*g*.sub.map(*m*1.sub.j)+β.Math.*g*.sub.map(*m*2.sub.j)]=*f*(*m*1.sub.j,*m*2.sub.j)

(104) The fact that two different couples of messages (m**1**.sub.i, m**2**.sub.i) and (m**1**′.sub.i, m**2**′.sub.i) always lead to two different values of the intermediate quantity α.Math.m**1**.sub.i+β.Math.m**2**.sub.i (respectively, α.Math.m**1**′.sub.i+β.Math.m**2**′.sub.i), due to the specific structure of S, is one of the elements that enables to apply the method to any, arbitrary two variable function f from T.sub.pi×T.sub.pi to T.sub.pi, or even from T.sub.pi×T.sub.pi to T.

(105) The encrypted version of f(m**1**.sub.i, m**2**.sub.i) is noted

c.sub.f,i=*F*(*c*1.sub.i,*c*2.sub.i)

(106) When c.sub.f,i is decrypted using the secret key s, it gives f(m**1**.sub.i, m**2**.sub.i).

(107) c.sub.f,i is determined, homomorphically, from the ciphertexts c**1**.sub.i and c**2**.sub.i (without decrypting c**1**.sub.i or c**2**.sub.i), according to the following equation:

c.sub.f,i=*F*(*c*1.sub.i, *c*2.sub.i)=*G*.sub.f[(α.Math.*G*.sub.map(*c*1.sub.i))⊕(β.Math.*G*.sub.map(*c*2.sub.i))]

where Gf denotes the bootstrapping function described above in the particular case where g is g.sub.f, and

where G.sub.map denotes the bootstrapping function described above, in the particular case where g is g.sub.map.

(108) It may be noted that the uncyphered values of α and β are used directly to compute c.sub.f,i. In other words, the coefficients α and β are not ciphered.
(109) This method can be applied to any function f (in particular, whether f is symmetric or not).
(110) When the function f is symmetric, α and β are chosen equal to each other. For instance, they may be chosen each as being equal to 1. In this case, c.sub.f,i is determined homomorphically from c**1**.sub.i and c**2**.sub.i according to the following equation (see FIG. **2**):
c.sub.f,i=*G*.sub.f[*G*.sub.map(*c*1.sub.i)⊕*G*.sub.map(*c*2.sub.i)]
(111) This way to compute c.sub.f,l is beneficial as there is no multiplication by α or β to be carried on, and as the noise terms are not amplified, as they are multiplied by 1.
(112) It may be noted that the Max(c**1**.sub.i, c**2**.sub.i) function, for instance, whose output is the cipher text c**1**.sub.i, or c**2**.sub.i, that is associated to the highest of the two messages m**1**.sub.i and m**2**.sub.i (highest, with respect to usual order of T.sub.pi values), is a symmetric function.
(113) When the function f is not symmetric, α and β are chosen different from each other, and the elements of S are such that, for any set of indexes (i,j,k,l) with (i,j)≠(k,l), α.Math.μ.sub.i+β.Math.μ.sub.j is different from α.Math.μ.sub.k+β.Math.μ.sub.l modulo ½, even when (k,l)=(j,i).
(114) In this case, α and β can be chosen as being equal to +1 and −1 respectively, for instance. c.sub.f,i is then determined homomorphically from c**1**.sub.i and c**2**.sub.i according to the following equation
c.sub.f,i=*G*.sub.f[*G*.sub.map(*c*1.sub.i)⊖*G*.sub.map(*c*2.sub.i)]
Again, this kind of implementation is beneficial, as it involves no multiplication by a coefficient higher than 1, and so no noise amplification.
(115) This method for evaluating homomorphically a two-variable function has been presented above in a case where the function f is a function from T.sub.pi×T.sub.pi to T.sub.pi, or from T.sub.pi×T.sub.pi to T. Still, it can be implemented identically when the two variables of the function f each belongs to a discrete set of p.sub.i elements (possibly different from T.sub.pi, but consisting of p.sub.i elements, with pi odd, and such that twice the difference between any two of the elements of this set is not an integer number), the values of function f belonging to T.
(116) Carry-Less Processing of a Message m, by Decomposing it into Orthogonal Sub-Messages m.sub.i.
(117) As mentioned above, the disclosed technology also concerns a technique for encrypting and processing 'very long' messages x (or m), using a specific decomposition of the message x into smaller messages x.sub.i, this decomposition being based on the Chinese remainder theorem decomposition and enabling to process the sub-messages x.sub.i, or, for instance, their counterparts x.sub.i/p.sub.i, separately from each other, without having to take into account possible carries.
(118) This technique can be applied to any message belonging to custom character.sub.p=custom character/pcustom character, p being equal to Π.sub.i=1.sup.rp.sub.i, where the integer numbers p.sub.i are pairwise coprime.
(119) More precisely, for any message x belonging to custom character.sub.p, the corresponding encrypted message C is determined by executing the following operations (FIG. **3**): Step D: decomposing x into its components x.sub.i, i=1 . . . r, with x.sub.i equal to x modulo p.sub.i (the Chinese remainder theorem stating that such a decomposition is always possible), Step E: for each component x.sub.i whose associated factor p.sub.i is odd, determining an encrypted version c.sub.i of a message m.sub.i, according to the encryption method presented above, m.sub.i being, among the elements of the discrete set of p.sub.i elements mentioned above (e.g.: T.sub.pi), the element that is associated to x.sub.i by a given bijective relationship between custom character.sub.pi and said discrete set, if a component x.sub.i is associated to a factor p.sub.i equal to 2, determining an encrypted version c.sub.i of a quantity Q.sub.i associated to x.sub.i and that can takes two distinct values, c.sub.i being equal to (a.sub.i,b.sub.i) with b.sub.i=Q.sub.i+e.sub.i+a.sub.i.Math.s, returning C=(c.sub.1, . . . c.sub.i, . . . c.sub.r).
(120) Below, we consider the case in which the discrete set consisting of p.sub.i distinct elements is T.sub.pi. The component x.sub.i may be associated (bijectively) to a corresponding element m.sub.i of T.sub.pi according to the following equation, for instance: m.sub.i=x.sub.i/p.sub.i, or according to the following equation m.sub.i=x.sub.i/p.sub.i, or, alternatively, according to the following equation m.sub.i=(x.sub.i+1)/p.sub.i.
(121) Besides, we consider below that the same secret key s is used on the different fields custom character.sub.p1, . . . , custom character.sub.pi, . . . , custom character.sub.pr. In other words, here, the same secret key s is used to cipher the different messages m.sub.1, . . . m.sub.i, . . . m.sub.r. Still, in alternative, different keys could be used to cipher the different (sub-) messages m.sub.1, . . . m.sub.i, . . . m.sub.r.
(122) In may be noted that, in other embodiments, the discrete set in question may be another set than T.sub.pi, and that another operation than the one mentioned above could be used to map custom character.sub.pi onto this discrete set (for instance, mi could be obtained by multiplying x.sub.i by any integer number that is coprime with p.sub.i, and then be divided by p.sub.i).
(123) Besides, here, the decomposition of x is carried on such that the only even factor p.sub.i, should there be one, is two. So, except from, possibly, a factor two, all the other factors pi are odd. And so, their corresponding messages m.sub.i can all be processed, individually, according to the methods described in detail above, for elements belonging to T.sub.pi, except, possibly, for one of them, associated to a factor p.sub.i equal to two, which is processed, for instance, according to existing, known techniques for binary messages.
(124) When processing such an encrypted message C=(c.sub.1, . . . c.sub.i, . . . c.sub.r), or when processing a couple (C**1**, C**2**) of two such messages C**1**=(*c***1**.sub.1, . . . c**1**.sub.i, . . . c**1**.sub.r) and C**2**=(*c***2**.sub.1, . . . c**2**.sub.i, . . . c**2**.sub.r), the components c.sub.1, . . . c.sub.i, . . . c.sub.r of C, or the couples of components (c**1**.sub.1, c**2**.sub.1) . . . (c**1**.sub.i, c**2**.sub.i), . . . (c**1**.sub.r, c**2**.sub.r) are processed, homomorphically, independently from each other (that is, without taking into account any carry).
(125) For instance, the homomorphic addition of two such messages C**1** and C**2** is achieved as follow (see FIG. **4**):
*C*1⊕*C*2=(*c*1.sub.1⊕*c*2.sub.1, . . . , *c*1.sub.i⊕*c*2.sub.i, . . . *c*1.sub.r⊕*c***2**.sub.r)
Similarly, the homomorphic multiplication of two such messages C**1** and C**2** is achieved as follow, based on the T.sub.pi×T.sub.pi homomorphic multiplication ⊙ presented above (see FIG. **5**):
*C*1⊙*C*2=(*c*1.sub.1⊙*c*2.sub.1, . . . , *c*1.sub.i⊙*c*2.sub.i, . . . *c*1.sub.r⊙*c*2.sub.r)
Similarly, a refreshed version C′ of such an encrypted message C=(c.sub.1, . . . c.sub.i, . . . c.sub.r) is determined by computing the following quantity C′=(G(c.sub.1), . . . G(c.sub.i), . . . , G(c.sub.r)).
Electronic Devices and System
(126) FIG. **6** represents schematically a cryptographic system **1** comprising an encrypting device **2** and a processing device **3**, for processing encrypted messages without decrypting them.
(127) The encrypting device **2** and the processing device **3** comprise each at least one processor, and at least one memory.
(128) The encrypting device **2** is programmed, or otherwise arranged for executing the following steps: receiving data representative of an unencrypted message m.sub.i, that belongs to a discrete set consisting of p.sub.i distinct elements, with p.sub.i being an odd number, said discrete set being the discrete torus
(129) $T_{\mathrm{pi}} = \frac{1}{p_i}\{-\frac{p_i-1}{2}, -\frac{p_i-1}{2}+1, \text{ .Math. }, \frac{p_i-1}{2}-1, \frac{p_i-1}{2}\}$,
or the set T.sub.pi,N[X] of polynomials of degree N−1 whose coefficients belong to T.sub.pi, or to another discrete set in bijective relationship with T.sub.pi, processing said data according to the method for encrypting such messages, that has been presented above, to determine encrypted data representative of the encrypted message c.sub.i, the secret key s employed during said processing, for instance stored, at least temporarily, in the memory of the encrypting device, being accessed by the processor or processors during said processing.
(130) The processing device **3**, which is distinct from the encrypting device **2**, is configured for receiving the encrypted message c.sub.i from the encrypting device **2**, and, possibly, to receive other such encrypted messages.
(131) The processing device **3** is programmed, or otherwise arranged to execute the following steps: receiving data, representative of the encrypted

message c.sub.i produced by the encrypting device **2**, processing said data, according to the method for bootstrapping described above, to determine refreshed data, representative of the encrypted message c′.sub.i, said processing being achieved without decrypting c.sub.i and without using, reading or accessing the secret key s.
(132) In particular, during this bootstrapping, the unencrypted message m.sub.i corresponding to c.sub.i does not appear nor result from an operation, nor it is used, read or accessed. The unencrypted message m.sub.i, just as the secret key s, thus remains absent from the processing device **3** at all time.
(133) The processing device could also be programmed to execute the homomorphic multiplication method presented above and/or to homomorphic evaluate of a two-variable function like the one presented above.
(134) The processing device **3** could also be programmed to execute one or several of the homomorphic processing methods for encrypted messages like the message C=(c.sub.1, . . . c.sub.i, . . . c.sub.r), that has been presented above. And the encrypting device **2** could be programmed to produce such encrypted messages, from unencrypted messages x belonging to Z.sub.p.

Claims

1. Method for determining an encrypted message c.sub.i, which is an encrypted version of a message m.sub.i, c.sub.i being equal to (a.sub.i,b.sub.i) where b.sub.i is determined from m.sub.i and from a secret key s, b.sub.i being equal to m.sub.i+e.sub.i+a.sub.i.Math.s with a.sub.i being a randomly selected vector for projecting s and e.sub.i being a random noise component added to m.sub.i+a.sub.i.Math.s, wherein the message m.sub.i belongs to a discrete set consisting of p.sub.i distinct elements, with p.sub.i being an odd number, the elements of said discrete set being such that twice the difference between any two of these elements is not an integer number, said discrete set being the discrete torus
$T_{\mathrm{pi}} = \frac{1}{p_i}\{-\frac{p_i - 1}{2}, -\frac{p_i - 1}{2} + 1, \text{ .Math. }, \frac{p_i - 1}{2} - 1, \frac{p_i - 1}{2}\}$, or the set T.sub.pi,N[X] of polynomials of degree N−1 whose coefficients belong to T.sub.pi, or another discrete set in bijective relationship with T.sub.pi.
2. A Method for bootstrapping the encrypted message c.sub.i=(a.sub.i,b.sub.i) determined according to claim 1, the method for bootstrapping comprising a determination of a refreshed encrypted message c.sub.i′=(a.sub.i′,b.sub.i′), which is an encrypted version of a value g(m.sub.i) of a function g applied to the message m.sub.i, c.sub.i′ having a noise component e.sub.i′ smaller than e.sub.i.
3. The method according to claim 2, wherein the determination of c.sub.i′ comprises a step of determining homomorphically the constant coefficient coef.sub.0(X.sup.q.Math.W.sub.g(X)) of X.sup.q.Math.W.sub.g(X) mod (X.sup.N+1), where W.sub.g(X)=Σ.sub.j=0.sup.N-1w.sub.j.Math.X.sup.j is a polynomial of degree N−1 whose coefficients w.sub.j are given by the following formula:
(−1).sup. ⌐ └2Nm.sup.i.sup.┐ /N┐ .*Math.w*.sub.j=*g*(*m*.sub.i), where $j = N \lceil \lfloor 2Nm.\mathrm{sub}.i \rfloor / N \rceil - \lfloor 2Nm.\mathrm{sub}.i \rfloor$ with └2 Nm.sub.i┐ being the integer number that is the nearest to 2 Nm.sub.i and with ⌐ └2Nm.sub.i┐ /N┐ being the ceiling function applied to └2Nm.sub.i┐ /N, and where q is the integer number that is the nearest to 2N(b.sub.i−a.sub.i.Math.s): q= └2N(b.sub.i−a.sub.i.Math.s)┐ .
4. The method according to claim 3, wherein N is below 50 times p.sub.i, or even below 30 times p.sub.i.
5. The method according to claim 3, wherein Nis above p.sub.i, or even above 5 times p.sub.i.
6. Method for homomorphically multiplying two messages m**1**.sub.i and m**2**.sub.i, each belonging to the discrete torus T.sub.pi or to the set of polynomials T.sub.pi,N[X], the method comprising a determination of an encrypted message c**3**.sub.i which is an encrypted version of the product of m**1**.sub.i by m**2**.sub.i, c**3**.sub.i being determined from encrypted messages c**1**.sub.i and c**2**.sub.i without decrypting c**1**.sub.i or c**2**.sub.i, c**1**.sub.i and c**2**.sub.i being encrypted versions of m**1**.sub.i and m**2**.sub.i respectively, that have been determined according to the method of claim 1, the determination of c**3**.sub.i comprising homomorphically determining
$\frac{(p_i + 1)}{2}$ .Math. $[\frac{(p_i + 1)}{2}$ .Math. p_i .Math. $(m1_i + m2_i)^2$ - $\frac{(p_i + 1)}{2}$ .Math. p_i .Math. $(m1_i - m2_i)^2]$ Where p.sub.i.Math.(m**1**.sub.i+m**2**.sub.i).sup.2 and p.sub.i.Math.(m**1**.sub.i−m**2**.sub.i).sup.2, or $\frac{(p_i + 1)}{2}$ .Math. p_i .Math. $(m1_i + m2_i)^2$ and $\frac{(p_i + 1)}{2}$ .Math. p_i .Math. $(m1_i - m2_i)^2$, or $[\frac{(p_t + 1)}{2}]^2$ .Math. p_i .Math. $(m1_i + m2_i)^2$ and $[\frac{(p_t + 1)}{2}]^2$ .Math. p_i .Math. $(m1_i - m2_i)^2$ are determined homomorphically by executing the method for bootstrapping according to any of claims 2 to 5, the function g being the function g.sub.sq′: m.sub.i.fwdarw.p.sub.i.Math.(m.sub.i).sup.2 mod (1), or $g_{\mathrm{sq}} : m_i$ .fwdarw. $\frac{(p_i + 1)}{2}$ .Math. $[p_i$ .Math. $(m_i)^2 \bmod(1)]$, or, respectively, $g_{\mathrm{sq}''} : m_i$ .fwdarw. $[\frac{(p_i + 1)}{2}]^2$ .Math. $[p_i$ .Math. $(m_i)^2 \bmod(1)]$.
7. Method for determining an encrypted message c.sub.f,i, which is an encrypted version of a value f(m**1**.sub.i,m**2**.sub.i) of a two-variable function f applied to messages m**1**.sub.i and m**2**.sub.i, m**1**.sub.i and m**2**.sub.i belonging each to said discrete set, c.sub.f,i being determined from encrypted messages c**1**.sub.i and c**2**.sub.i without decrypting c**1**.sub.i or c**2**.sub.i, c**1**.sub.i and c**2**.sub.i being encrypted versions of m**1**.sub.i and m**2**.sub.i respectively, that have been determined according to the method of claim 1, the determination of c.sub.f,i comprising: determining c**1**′.sub.i and c**2**′.sub.i, from c**1**.sub.i and c**2**.sub.i respectively, by executing the method for bootstrapping, the function g employed during this bootstrapping being a function g.sub.map that maps said discrete set onto a set S whose elements μ.sub.j belong each to the torus T=]−1/2,1/2] and are such that, for any set of indexes (i,j,k,l) with (i,j)≠(k,l), (k,l) being different from (j,i): α.Math.μ.sub.i+β.Math.μ.sub.j is different from α.Math.μ.sub.k+β.Math.μ.sub.l modulo 1/2, with α and β being two constant and integer, given numbers, c**1**′.sub.i and c**2**′.sub.i being the encrypted versions of m**1**.sub.i=g.sub.map(m**1**.sub.i) and m**2**.sub.i=g.sub.map(m**2**.sub.i) respectively, where m**1**.sub.i and m**2**.sub.i belong each to S, determining homomorphicically, from c**1**′.sub.i and c**2**′.sub.i, an encrypted message c.sub.s which is an encrypted version of α.Math.m**1**.sub.i+β.Math.m**2**.sub.i, determining c.sub.f,i from c.sub.s by executing the method for bootstrapping according to any of claims 2 to 5, the function g employed during this bootstrapping being a one-variable function g.sub.f defined by the following condition: for any couple of messages m**1**.sub.j, m**2**.sub.j belonging each to said discrete set,
g.sub.f(α.Math.*m*1.sub.j+*β.Math.m*2.sub.j)=*f*(*m*1.sub.j,*m*2.sub.i) that is:
g.sub.f*[α.Math.g*.sub.map(*m*1.sub.i)+β.Math.*g*.sub.map(*m*2.sub.i)]=*f*(*m*1.sub.i,*m*2.sub.i).
8. The method according to claim 7, wherein the elements of the set S are such that for any set of indexes (i,j,k,l) with (i,j)≠(k,l), α.Math.μ.sub.i+β.Math.μ.sub.j is different from α.Math.μ.sub.k+β.Math.μ.sub.l modulo 1/2, even when (k,l)=(j,i).
9. The method according to claim 8, wherein α=1 and B=−1, c.sub.s being determined by computing the homomorphic difference Θ between c**1**′.sub.i and c**2**′: c.sub.s=c**1**′.sub.i⊖c**2**′.sub.i, or wherein α=−1 and β=1, c.sub.s being determined by computing the homomorphic difference Θ between c**2**′.sub.i and c**1**′: c.sub.s=c**2**′.sub.i⊖c**1**′.sub.i.
10. The method according to claim 7, wherein the function f is symmetric, f(m**1**.sub.i,m**2**.sub.i) being equal to f(m**2**.sub.i,m**1**.sub.i) for any couple of messages m**1**.sub.i and m**2**.sub.i, and wherein α and β are each equal to 1, c.sub.s being determined by computing the homomorphic sum Θ of c**1**′.sub.i and c**2**′: c.sub.s=c**1**′.sub.i⊖c**2**′.sub.i.
11. The method for determining an encrypted message C, which is an encrypted version of a message x belonging to custom character.sub.p= custom character/p custom character, p being equal to Π.sub.i=1.sup.rp.sub.i where the integer numbers p.sub.i are pairwise coprime, the method comprising: (D) decomposing x into its components x.sub.i, i=1 . . . r, with x.sub.i equal to x modulo p.sub.i, (E) for each component x.sub.i whose associated factor p.sub.i is odd, determining an encrypted version c.sub.i of a message m.sub.i, according to the method of claim 1, m.sub.i being, among the elements of said discrete set, the element that is associated to x.sub.i by a given bijective relationship between custom character.sub.pi and said discrete set, if a component x.sub.i is associated to a factor p.sub.i equal to 2, determining an encrypted version c.sub.i of a quantity Q.sub.i associated to x.sub.i and that can takes two distinct values, c.sub.i being equal to (a.sub.i,b.sub.i) with b.sub.i=Q.sub.i+e.sub.i+a.sub.i.Math.s,

returning C=(c.sub.1, . . . c.sub.i, . . . c.sub.r).

12. The method for bootstrapping the encrypted message C=(c.sub.1, . . . c.sub.i, . . . c.sub.r) determined according to claim 11, said method comprising: for each c.sub.i whose associated factor p.sub.i is odd, determining a refreshed encrypted message c′.sub.i, from c.sub.i, by executing the method for bootstrapping, returning C′=(c′.sub.1, . . . , c′.sup.i, . . . c′.sup.r).

13. The method for determining an encrypted message C**3**=(*c***3**.sub.1, . . . c**3**.sub.i, . . . c**3**.sub.r), which is an encrypted version of the product of two messages x**1** and x**2**, each belonging to custom character.sub.p, C**3** being determined from encrypted messages C**1** and C**2**, without decrypting C**1** or C**2**, C**1** and C**2** being encrypted versions of x**1** and x**2** respectively, C**1**=(*c***1**.sub.1, . . . c**1**.sub.i, . . . c**1**.sub.r) and C**2**=(*c***2**.sub.1, . . . c**2**.sub.i, . . . c**2**.sub.r) having been determined from x**1** and x**2** respectively, according to the method of claim 11, wherein said discrete set is T.sub.pi or T.sub.pi,N[X], wherein each component c**3**.sub.i of C**3** is determined from the corresponding components c**1**.sub.i and c**2**.sub.i of C**1** and C**2** when c**1**.sub.i and c**2**.sub.i are associated to a factor p.sub.i that is odd.

14. Encrypting device, comprising one or more processors and at least one memory, the encrypting device being programmed to make the processor or processors executing the following steps: receiving data representative of an unencrypted message m.sub.i, that belongs to a discrete set consisting of p.sub.i distinct elements, with p.sub.i being an odd number and wherein the elements of said discrete set are such that twice the difference between any two of these elements is not an integer number, said discrete set being the discrete torus $T_{\text{pi}} = \frac{1}{p_i}\{-\frac{p_i - 1}{2}, -\frac{p_i - 1}{2} + 1,$ .Math. , $\frac{p_i - 1}{2} - 1, \frac{p_i - 1}{2}\},$ or the set T.sub.pi,N[X] of polynomials of degree N−1 whose coefficients belong to T.sub.pi, or to another discrete set in bijective relationship with T.sub.pi, processing said data according to the method of claim 1, to determine encrypted data representative of the encrypted message c.sub.i, the secret key s employed during said processing, stored at least transitorily in the memory of the encrypting device, being accessed by the processor or processors during said processing.

15. Cryptographic system comprising the encrypting device of claim 14 and a processing device, the processing device comprising one or more processors and at least one memory, the processing device being programmed to make the processor or processors executing the following steps: receiving data, representative of the encrypted message c.sub.i that has been produced by the encrypting device, processing said data, according to a method for bootstrapping, to determine refreshed data, representative of the encrypted message c′.sub.i, said processing being achieved without decrypting c.sub.i and without using, reading or accessing the secret key s.

16. A non-transitory computer-readable medium comprising instructions that, when executed by a computer, make the computer to execute the bootstrapping method according to claim 2.

17. A non-transitory computer-readable medium comprising instructions that, when executed by a computer, make the computer to execute the encrypting method according to claim 11.